- Integrate the model into a web or mobile application for real-world usage.
- Use platforms like TensorFlow Serving or TorchServe for efficient model serving.

7. Scalability and Optimization:
    - Optimize the model for inference using TensorRT or ONNX.
    - Use cloud platforms like AWS, GCP, or Azure for scalable deployment.
    - Enable batch processing for handling multiple requests simultaneously.

Outcome:

By implementing this project, you will build a powerful image classification system that can be applied to various industries, including manufacturing (quality control), healthcare (disease detection), and retail (product categorization). The project demonstrates the capabilities of CNNs and transfer learning in solving real-world problems.

Example Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout from tensorflow.keras.preprocessing.image import ImageDataGenerator # Step 1: Load and Preprocess Data
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.cifar10.load_data()
train_images, test_images = train_images /255.0, test_images 255.0

# Step 2: Data Augmentation
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(train_images)

# Step 3: Build CNN Model
model = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)), MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'), MaxPooling2D((2, 2)),
```